1. Requirement

Functional:

- Admin should be able to create or edit policies
- Employee should be able to see their balance and request time off for specific date and hour
- Admin should be able to approve or deny req and balance update to match. Employee can cancel
- Balances should accure automatically on schedule or from payroll hours worked
- Also designed for - custom work hours, holiday calendar, carryover and expiration, tenure tiers

Non-functional:

- Correct under retries and concurrency: no double accrual, no double spend, strong consistency on balance
- Auditable: every balance change and every policy, alignment and request change records who, when and why.
- Policy edits never change past result

2. Core Entities

- **Policy** - Belongs to company, has a type (vacation, sick)
- **Policy version** - Immutable. effective-dated rules mode, accrual, limits, year-end, tenure tiers
- **Policy Alignment**: Employee to policy, with effective dates
- **Time off request**: Employee, type, reason, status, chargeable hours, with one RequestDay row per date
- **Ledger Entry**: Append-only: accrual, usage, reversal, adjustment, expiration. Balance is derived from it

Admin
`POST /v1/policies`
Create a policy with its first version

Admin
`POST /v1/policies/{id}/versions`
Change rules with an `effectiveFrom`, creating a new version

Admin
`POST /v1/policies/{id}/assignments`
Assign employees (`employeeIds`, `effectiveFrom`)

Admin
`DELETE /v1/policies/{id}/assignments/{employeeId}`
End an assignment (sets `effective_to`)

Admin
`GET /v1/requests?status=PENDING`
Approval queue

Admin
`POST /v1/requests/{id}/approve` and `/deny`
Decide, deny takes a note

Admin
`POST /v1/employees/{id}/adjustments`
Manual hours, reason required

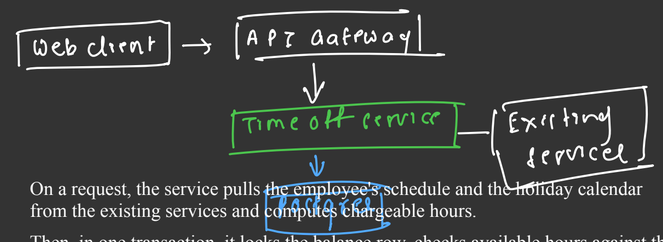
Employee
`GET /v1/me/balances`
Per policy: balance, pending, available

Employee
`POST /v1/requests`
Body `{typeId, reason, days: [{date, hours?}]}` with an `Idempotency-Key` header. Returns a PENDING request with chargeable hours

Employee
`POST /v1/requests/{id}/cancel`
Cancel while pending or approved

Both
`GET /v1/employees/{id}/ledger?policyId=`
History behind a balance (employees see only their own)

4. High level Design



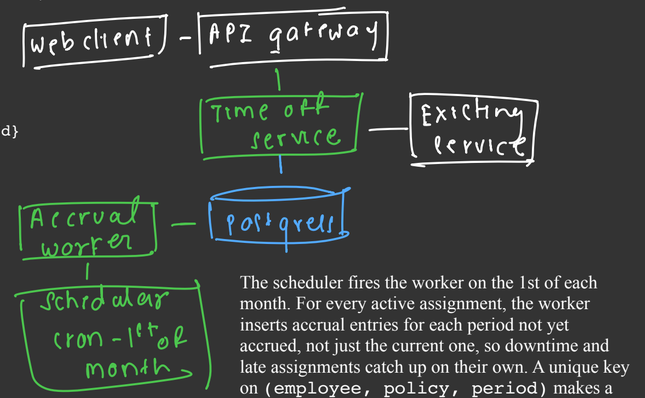
On a request, the service pulls the employee's schedule and the holiday calendar from the existing services and computes chargeable hours.

Then, in one transaction, it locks the balance row, checks available hours against the policy floor, and inserts the request as pending.

Approval re-checks the balance and, in one transaction, writes the usage entry, flips the status and records the audit event.

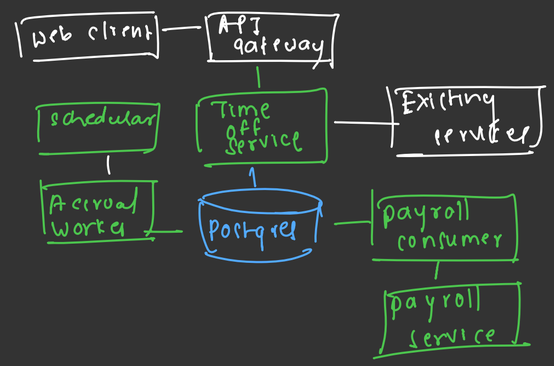
Everything lives in Postgres, and the balance is derived from the ledger.

Step 2: time-based accrual



The scheduler fires the worker on the 1st of each month. For every active assignment, the worker inserts accrual entries for each period not yet accrued, not just the current one, so downtime and late assignments catch up on their own. A unique key on `(employee, policy, period)` makes a rerun harmless. It turns "15 days a year" into hours using each employee's work schedule from the existing services. Mid-period joins are prorated by a hook that runs when an assignment is created.

Step 3: hour-worked accrual (requirement 4, second half)



Payroll fires `on_payroll_processed` onto a queue, and the consumer converts hours worked into accrual entries using the policy's ratio. For their literal example, "1 hour per 3 days worked", that's 1h per 24h worked at 8h days. A unique key on `(payroll_run_id, employee, policy)` stops a redelivered event from paying twice. The queue absorbs the burst after each payroll run, and it needs a dead-letter queue plus a reconciliation job to catch any event that never arrives.

1. Concurrency on the balance (the lock, submit vs approve races)
2. Idempotent accrual (retries, redelivery, payroll corrections)
3. Policy versioning (effective dates, forward-only changes, retroactive adjustments)
4. Auditability (append-only ledger, request and assignment history)
5. The bonus challenges (work hours, holidays, carryover, tenure)